

Traffic Planning Mission - Algorithm Report

1. Introduction

This report documents the design and implementation of program **Traffic Planning** by setting up a **system of linear equation**. The program solves the **system of linear equation** from a file by the **Gauss-Jordan elimination method**.

2. Algorithm Specifications

2.1 Require

A **system of linear equation** given in a plain text file with the following format:

- The first line contains a positive number **n**, the number of unknowns.
- The next **n** lines, each contains **n+1** space-separated **double** values representing an **augmented matrix**.

2.2 Ensure

Print the answer with the following format:

- **value_1 ... value_n** iff the **system** has a **unique solution**.
- **no unique solution** iff the **system** has no a **unique solution**.

2.3 Key Data Structures

- An integer **n** represents the size of matrix.
- A 2D-array of **double a** represents the augmented matrix.

3. Implementation Details

3.1 Core Function

slice(str sentence , int i , int j)

- **Purpose:** To slice a string from index **i** to **j-1** so that we can directly use the function **atof()**.
- **Variables:**
 - **sentence:** The string that needs to be cut.
 - **i** and **j**: The index.
- **Implementations:** Claims a new string **result** and assigns it one by one in a for loop.
- **Pseudocode:**

```
fun slice( sentence : str , i : int , j : int ) -> result : str
!require : 0 <= i <= j <= len( sentence )
  length := len( sentence )
  result := new ( str , j - i + 1 )
  for every k in [ i , j ) do
```

```

        result[k - i] := sentence[k]
        result[j - i] := '\0'
    !ensure : result = the substring from sentence[i] to sentence[j - 1]

```

split(str sentence)

- **Purpose:** To split a string by space.
- **Variables:**
 - **sentence:** The string that needs to be split.
 - **length:** The length of string **sentence**.
- **Implementaion:** Counts the number of the white space and add one to confirm the number of the substring. Each new substring is from the position of last white space plus one to the position of next white space minus one. Then uses the function **slice()** to cut the new substring and stores in a new array of **string**.
- **Pseudocode:**

```

fun split( sentence : str ) -> result : arr( str )
!require : sentence != NULL
    length := len( sentence )
    count := 1
    for every i in [ 0 , length ) do
        if sentence[i] = ' ' then
            count := count + 1
    result := new( str , count )
    l := 0
    r := -1
    for every i in [ 0 , count ) do
        l := r + 1
        r := search( sentence , l , ' ' )
        result[i] := slice( sentence , l , r )
    !ensure : len( result ) = no. of white space in sentence + 1

```

input(arr(arr(a)) , file f , int n)

- **Purpose:** To input the augmented matrix from file **f**
- **Variables:**
 - **a:** The 2D-array of **double** to store the augmented matrix.
 - **f:** The file documenting the input data.
 - **n:** The size of augmented matrix.
 - **line:** An string that is the current row of matrix.
 - **tokens:** An array of **string** storing the splited row of matrix.
- **Implementation:** Reads **n** lines from the file **f**. For each line, allocates memory for **n+1 double** to the array **a**. Then split the line into individual **tokens**. Next, converts each token into **double** by function **atof()**.
- **Pseudocode:**

```

fun input( a : arr( arr( double ) ) , f : file , n : int )
!require : n > 0
  line = "\0"
  for every i in [ 0 , n ) do
    a[i] := new( double , n+1 )
    line := get_next_line( f )
    tokens := split( line )
    for every j in [ 0 , n ] do
      a[i][j] := atof( tokens[j] )
!ensure : a is an n*(n+1) augmented matrix filled with the value from
file f

```

interchange(arr(arr(a)) , int n)

- **Purpose:** To interchange rows to avoid zero pivot elements.
- **Variables:**
 - **a**: The 2D-array of **double** to store the augmented matrix.
 - **n**: The size of augmented matrix.
- **Implementation:** Checks each **a[i][i]** in each row. If it equals to zero, then find the elements in the same column of the following rows which is non-zero and then swap them.
- **Pseudocode:**

```

fun interchange( a : arr( arr( double ) ) , n : int )
!require : a is an n*(n+1) augmented matrix ; n > 0
  for every i in [ 0 , n-1 ) do
    found := 0
    if a[i][i] = 0 then
      for every j in [ i+1 , n ) do
        if not a[j][i] = 0 then
          temp := a[i]
          a[i] := a[j]
          a[j] := temp
          found := 1
          break
    if found then
      continue
!ensure : for every i in [ 0 , n-1 ) , if exists j > i with a[j][i] !=
0 then a[i][i] != 0

```

rank(arr(arr(a)) , int n)

- **Purpose:** To calculate the rank of the augmented matrix.
- **Variables:**
 - **a**: The 2D-array of **double** to store the augmented matrix.
 - **n**: The size of augmented matrix.
 - **rank**: The rank of the augmented matrix.
 - **all_zero**: A bool to mark if the row is all zero.

- **Implementation:** Let the rank be the size of matrix as default. Then checks each row. If it is all zero, which means this row has no pivot, then rank decrease by one.
- **Pseudocode:**

```

fun rank( a : arr( arr( double ) ) , n : int ) -> rank : int
!require : a is an n*(n+1) augmented matrix ; n > 0
  rank := n
  for every i in [ 0 , n ) do
    all_zero := true
    for every j in [ 0 , n ) do
      if not a[i][j] = 0 then
        all_zero := false
    if all_zero then
      rank := rank - 1
!ensure : 0 <= rank <= n

```

has_unique_solution(arr(arr(double)) a , int n)

- **Purpose:** To check if the matrix has a unique solution.
- **Variables:**
 - **a**: The 2D-array of **double** to store the augmented matrix.
 - **n**: The size of augmented matrix.
 - **r**: The rank of the augmented matrix.
- **Implementation:** By calculating the rank of the to judge if the system has unique solution.
- **Pseudocode:**

```

fun has_unique_solution( a : arr( arr( double ) ) , n : int ) ->
result : bool
!require : a is an n*(n+1) augmented matrix ; n > 0
  r := rank( a , n )
  if r < n then
    result := false
  else
    result := true
!ensure : has_unique_solution( a , n ) iff the augmented matrix has
unique solution

```

REF(arr(arr(double)) a , int n)

- **Purpose:** To convert the matrix to REF(Row Echelon Form) form.
- **Variables:**
 - **a**: The 2D-array of **double** to store the augmented matrix.
 - **n**: The size of augmented matrix.
 - **pivot**: The pivot of the current row.
 - **factor**: The multiplier used to eliminate elements below the pivot in the current column.

- **Implementation:** For each row i , takes the diagonal element $a[i][i]$ as the pivot. If the pivot is non-zero, normalizes the entire row by dividing all elements in row i from column i to $n+1$ by the pivot. Then, for each subsequent row j below row i , calculates a factor equal to the element $a[j][i]$ that is the element directly below the pivot, and subtracts factor times row i from row j for all columns from i to $n+1$. This creates zeros in the i -th column below the i -th row, progressing towards upper triangular form. The process repeats for each row until reaching the last row.
- **Pseudocode:**

```

fun REF( a : arr( arr( double ) ) , n : int )
!require : a is an n*(n+1) augmented matrix ; n > 0
  for every i in [ 0 , n ) do
    pivot := a[i][i]
    if not pivot = 0 then
      for every j in [ i , n ] do
        a[i][j] = a[i][j] / pivot
      for every j in [ i + 1 , n ) do
        factor = a[j][i]
        for every k in [ i , n ] do
          a[j][k] = a[j][k] - factor * a[i][k]
!ensure : a is in row echelon form

```

RREF(arr(arr(double)) a , int n)

- **Purpose:**
- **Variables:**
 - **a**: The 2D-array of **double** to store the augmented matrix.
 - **n**: The size of augmented matrix.
 - **factor**: The multiplier used to eliminate elements above the pivot in the current column.
- **Implementation:** Initially, uses the function **REF()** to convert the matrix to REF. Then performs backward elimination starting from the bottom row and moving upward. For each row i from $n-1$ down to 0 , it processes all rows above it (rows j from $i-1$ down to 0). For each row j above row i , calculates factor as $a[j][i]$ (the element in the i -th column of row j), then subtracts factor times row i from row j for all columns from n down to i . This elimination process creates zeros above each pivot, resulting in a reduced row echelon form where each pivot column contains only the pivot element as 1 , with all other elements being 0 .
- **Pseudocode:**

```

fun RREF ( a : arr( arr( double ) ) , n : int )
!require : a is an n*(n+1) augmented matrix ; n > 0
  REF( a , n )
  for every i in [ n - 1 , 0 ] with i reduced do
    for every j in [ i - 1 , 0 ] with i reduced do
      factor := a[j][i]
      for every k in [ n , i ] with i reduced do
        a[j][k] := a[j][k] - factor * a[i][k]
!ensure : a is an n*(n+1) augmented matrix in reduced row echelon form

```

3.2 Key Decisions

3.2.1 String Processing

```
line = get_next_line( f ) ;  
arr( str ) tokens = split( line ) ;
```

By splitting the string by space, we can directly use the function `atof()`, which make the code more simple.

3.2.2 Solution judgement

We judges if the system has unique solution by calculating `rank(a)` and compare it with the size `n`.

```
int r = rank( a , n ) ;  
if ( r < n ) return 0 ;  
else return 1 ;
```

In this way, we have a systematic method to judge if the system has unique solution.

4. Execution Trace Example

4.1 System with Unique Solution

Input Data

```
./gauss 1.txt
```

1.txt:

```
3  
2 -1 3 6  
1 4 0 -3  
2 6 -1 15
```

Input Processing

- Read `n = 3`
- Matrix initialized as:
 - Row 0: [2, -1, 3, 6]
 - Row 1: [1, 4, 0, -3]
 - Row 2: [2, 6, -1, 15]

Function `interchange()`

- Check if each diagonal elements is zero.
 - `a[0][0] == 2 != 0`
 - `a[1][1] == -1 != 0`
 - `a[2][2] == -1 != 0` No row interchange needed.

RREF converting

- `REF()`
 - `i = 0`
 - `pivot = a[0][0] = 2`
 - Divide row 0 by `pivot`
 - Row 0 becomes: [1, -0.5, 1.5, 3]
 - Eliminate below elements in column 0:
 - `j = 1` (for row 1): `factor = a[1][0] = 1`
 - Row 1 = Row 1 - 1 * Row 0 = [0, 4.5, -1.5, -6]
 - `j = 2` (for row 2): `factor = a[2][0] = 2`
 - Row 2 = Row 2 - 2 * Row 0 = [0, 7, -4, 9]
 - `i = 1`
 - `pivot = a[1][1] = 4.5`
 - Divide row 1 by `pivot`
 - Row 1 becomes: [0, 1, -1/3, -4/3]
 - Eliminate below elements in column 1:
 - `j = 2` (for row 2): `factor = a[2][1] = 7`
 - Row 2 = Row 2 - 7 * Row 1 = [0, 0, -5/3, 55/3]
 - `i = 2`
 - `pivot = a[2][2] = -5/3`
 - Divide row 2 by `pivot`
 - Row 2 becomes:[1, -11]
- Current matrix(REF):
 - [1, -0.5, 1.5, 3]
 - [0, 1, -1/3, -4/3]
 - [0, 0, 1, -11]
- `i = 2`: Eliminate column 2 in rows above
 - Row 1 (`j=1`): `factor = a[1][2] = -1/3`
 - Row 1 = Row 1 - (-1/3) × Row 2 = [0, 1, 0, -5]
 - Row 0 (`j=0`): `factor = a[0][2] = 1.5`
 - Row 0 = Row 0 - 1.5 × Row 2 = [1, -0.5, 0, 19.5]
- `i = 1`: Eliminate column 1 in rows above
 - Row 0 (`j=0`): `factor = a[0][1] = -0.5`
 - Row 0 = Row 0 - (-0.5) × Row 1 = [1, 0, 0, 22]
- `i = 0`: No rows above to process
- Final matrix(RREF):
 - [1, 0, 0, 17]
 - [0, 1, 0, -5]
 - [0, 0, 1, -11]

Judge unique solution

- `has_unique_solution()`
 - `r = rank( a , n ) = 3`
 - `return 1`

Output

```
17.0000000 -5.0000000 -11.0000000
```

4.2 System with No Solution

Input Data

```
./gauss 2.txt
```

2.txt:

```
3
1 2 3 6
4 5 6 15
7 8 9 25
```

Input Processing

- Read `n = 3`
- Matrix initialized as:
 - Row 0: [1, 2, 3, 6]
 - Row 1: [4, 5, 6, 15]
 - Row 2: [7, 8, 9, 25]

Function `interchange()`

- Check diagonal elements:
 - `a[0][0] = 1 ≠ 0`
 - `a[1][1] = -3 ≠ 0` (after operations)
 - `a[2][2] = 0` (will be identified later) No initial interchange needed.

RREF Converting

1. REF() Execution

- `i = 0`: Same as before
- `i = 1`: Same as before

- $i = 2$:
 - Row 2 becomes: $[0, 0, 0, 1]$ (after elimination)

Matrix in REF:

- $[1, 2, 3, 6]$
- $[0, 1, 2, 3]$
- $[0, 0, 0, 1]$

Detection of Inconsistency

During RREF conversion:

- The last row represents: $0x + 0y + 0z = 1$
- This is impossible → **No solution**

Output

```
no unique solution
```

4.3 System with Infinitely Many Solutions

Input Data

```
./gauss 3.txt
```

3.txt:

```
3
1 2 3 6
2 4 6 12
3 6 9 18
```

Input Processing

- Read $n = 3$
- Matrix initialized as:
 - Row 0: $[1, 2, 3, 6]$
 - Row 1: $[2, 4, 6, 12]$
 - Row 2: $[3, 6, 9, 18]$

Function `interchange()`

- Check diagonal elements:

- $a[0][0] = 1 \neq 0$
- $a[1][1] = 0$ (will be zero after operations)
- $a[2][2] = 0$ (will be zero after operations) No initial interchange needed.

RREF Converting

1. REF() Execution

- **i = 0 (First Row)**
 - $\text{pivot} = a[0][0] = 1$
 - Normalize row 0: [1, 2, 3, 6]
 - Eliminate column 0:
 - Row 1: factor = 2 $\rightarrow$ [0, 0, 0, 0]
 - Row 2: factor = 3 $\rightarrow$ [0, 0, 0, 0]
- **i = 1 (Second Row)**
 - $\text{pivot} = a[1][1] = 0$
 - No non-zero pivot found in column 1
 - Continue to next column
- **i = 2 (Third Row)**
 - $\text{pivot} = a[2][2] = 0$
 - All zero row

Matrix in REF:

- [1, 2, 3, 6]
- [0, 0, 0, 0]
- [0, 0, 0, 0]

2. RREF() Execution

- Only one non-zero row remains
- **Final RREF Matrix:**
- [1, 2, 3, 6]
- [0, 0, 0, 0]
- [0, 0, 0, 0]

Analysis for Infinite Solutions

- Rank of coefficient matrix = 1
- Number of variables = 3
- Number of free variables = $3 - 1 = 2$
- **General Solution:**

$$x = 6 - 2y - 3z$$

$$y = \text{free variable (parameter)}$$

```
z = free variable (parameter)
```

Output

```
no unique solution
```

4.4 Edge Cases

Input Data

```
./gauss 4.txt
```

4.txt:

```
0
0
```

That is

- $n = 0$

Special judge

In main.c, we have the code:

```
if ( !n )
{
    printf( "no unique solution\n" ) ;
    return 0 ;
}
```

Now, $n == 0$, then $!n == \text{true}$.

Output

```
no unique solution
```

5. Conclusion

This report has detailed the design, implementation, and execution of a traffic planning algorithm based on solving systems of linear equations. The program successfully reads an augmented matrix from a file, applies the Gauss-Jordan elimination method to compute its Reduced Row Echelon Form (RREF), and determines whether a unique solution exists. Through careful design decisions—such as file-based input for practical data handling, systematic string tokenization for parsing, and rank-based analysis for solution uniqueness—the algorithm provides a robust and reliable tool. The implementation effectively handles the standard cases of unique, infinite, and no solutions, as well as edge cases, demonstrating both its functional correctness and structural clarity.